Ultron

Ecom Manager

Standard Operating Procedure

Purpose, capabilities, set-up, daily operation and troubleshooting

Item	Detail
Organisation	Bindal Exports Ltd
System	Ultron · Ecom Manager (AI e-commerce manager on Telegram)
Code repository	github.com/bindalbhikan/Ultron (release v0.1.0 and later)
Local folder	G:\My Drive\BINDAL EXPORTS\AGENT\AGENT-ULTRON-BINDAL
Telegram bot	@bindal_exports_bot
Document version	0.1, 25 October 2026
Replaces	the bindal-bots repository and its command-menu Telegram bot (retired)

Contents

#	Section
1	Purpose and scope
2	What Ultron does
3	How it works
4	Daily schedule (IST)
5	Using Ultron: the owner's procedure
6	Set-up from scratch
7	Running Ultron on your own machine
8	Changing things
9	Monitoring and troubleshooting
10	Security rules
11	Retiring bindal-bots: checklist
A	Appendix: secrets, variables and folder layout

1. Purpose and scope

Ultron is Bindal Exports' AI e-commerce manager. It runs the daily operations reports of the marketplace business (Myntra, AJIO, Flipkart, Amazon, Meesho, Nykaa, COCOBLU), checks every run, remembers what the owner teaches it, and answers the owner in plain words on Telegram. It replaces the earlier set-up, where a Telegram command menu started separate bots and a failed run only said "failed".

This SOP explains what Ultron is for, what it can and cannot do, how it is set up, how the owner uses it every day, and what to do when something goes wrong. It is written for the owner (MD), anyone who operates the system, and whoever sets it up again on a new machine or account.

Goals

- One place to talk to: every question and every report request goes to the same Telegram bot.
- No silent failures: every report run is checked, and only real problems reach the owner, with the cause and a next step.
- Nothing runs without the owner: anything Ultron would start needs an Approve tap.
- It learns: preferences, facts and known harmless errors are remembered.
- Free to run at current volumes: the model runs on Cloudflare Workers AI's free daily allocation; GitHub Actions minutes are free for a public repository.

Out of scope: Ultron never changes prices, discounts or stock values, never messages customers, marketplaces or couriers, never deletes anything, and handles no money.

2. What Ultron does

2.1 Talks to the owner

- Answers questions in plain words (English or Hinglish), e.g. "how many orders yesterday?", "where is order FN2511505146?", "which orders are due today?".
- Looks things up with its tools: OMSGuru orders and history, sale by channel, orders due or overdue on SLA, recent bot runs, their logs and the sheets they wrote.
- Proposes actions with Approve / Reject buttons (see 2.4); it never runs them by itself.
- Understands follow-ups: it keeps each chat's last 10 messages (up to 3 days).

2.2 Watches every report run

- After every report workflow finishes, the watcher looks at it. A plainly fine run (succeeded, usual duration, no ERROR in its log) costs nothing and sends nothing.
- An unusual run (failed, far slower than usual, errors in the log) is read by the model: the real cause in the log, what it affected, and the suggested next step go to Telegram.
- It uses its memory: an error the owner called harmless is treated as OK; a problem seen before is called recurring. Every alert it sends is remembered as an incident.

2.3 Remembers

Kind	Example	Used for
preference	"Send the sale report in units, not rupees"	always in front of the model
fact	"PPMP is the second Myntra account"	when a message matches it
known_issue	"A Myntra SPF cooldown of up to 150 min is harmless"	the watcher calls it OK
decision	"Raise AJIO SPF claims only on Wednesday and Saturday"	when a message matches it
incident	noted by the watcher for every alert	spotting recurring problems

The memory is kept in the Telegram worker's Cloudflare KV, never in git. Text that looks like a password, OTP, token, key, phone number or email is refused before it is stored.

2.4 Actions it may propose (each needs Approve)

Action	What runs	Inputs it may set
sale	Daily sale report / sheet picture	from, to, today, snapshot
vms	Myntra VMS report / ERP vs VMS / sheet picture	from, to, erp_only, snapshot
ajiotat	AJIO TAT master update	days, orders
ajiospf	AJIO SPF run (raises claims on AJIO)	dry_run
myntraspf	Myntra SPF master	-
returns	Return analysis of a month	month
ajioads	AJIO ads / TOT gap report	-
myntraotp	Myntra courier OTPs now	-
myntrareports	Myntra daily reports	date
invpush	OMSGuru Push Inventory on every channel	dry_run

Action	What runs	Inputs it may set
morning, sla, labels, stock, evening, weekly, deadstock, listings, mis, myntratat	the scheduled reports, run now	morning / mis: day

2.5 Runs the daily reports

Section 4 lists every automatic run. The reports themselves (Excel sheets on Google Drive, Telegram summaries) are unchanged from the earlier bots; they now live inside Ultron (`src/ultron/workings`).

3. How it works

Part	Where	Job
Telegram bot	@bindal_exports_bot	The owner's only screen: questions, answers, reports, alerts, Approve / Reject
Telegram worker	Cloudflare Worker (worker/src/worker.js + worker/agent.js)	Receives every Telegram message; starts the daily runs on time (cron); serves the model, approvals and memory to Ultron; keeps dashboard numbers and Myntra logins (KV "DASH")
Ultron	GitHub repo bindalbhikan/Ultron (Python package "ultron")	The agent (chat, watch, memory, approvals, tools) and the report workings
GitHub Actions	the Ultron repo's .github/workflows	Runs each report and each Ultron check / answer
Model	Cloudflare Workers AI, @cf/openai/gpt-oss-120b	Reads, decides, writes answers (free daily allocation)
Data	Google Drive sheets; OMSGuru, Myntra, AJIO, Flipkart, ERP, SRM, VMS portals	Where reports read from and write to
Dashboard	Cloudflare Worker (dashboard/)	Latest numbers of every report, sign-in from Telegram

3.1 A message from the owner

1. The owner writes to the bot. Telegram sends it to the worker, which checks the chat is allowed.
2. The worker starts the *Agent chat* workflow and replies "On it".
3. Ultron reads its memory, calls its tools (OMSGuru, runs, sheets), and the model writes the answer.
4. The answer arrives in the same chat, usually within 1 to 2 minutes.
5. If something should run, Ultron sends Approve / Reject. Only Approve starts that one workflow, with only the inputs shown; an approval works once and expires after 24 hours.

3.2 A scheduled report

1. A cron trigger fires (worker) or GitHub's schedule starts the workflow.
2. The report runs and posts its result to Telegram and its sheet to Drive.
3. Its log and output are packed and encrypted (the repo is public); only the encrypted file is kept.
4. When it finishes, *Agent watch* checks it; only a real problem reaches the owner.

4. Daily schedule (IST)

Time (IST)	Report	Started by
7:00 AM daily	SJIT and Myntra SPF report requests (portal queues them)	worker cron
7:00 AM on the 15th	Return analysis report request	worker cron
9:00 AM daily	Myntra SPF master; AJIO SPF (tickets + replies Wed, Sat; master only other days)	worker cron
9:00 AM on the 3rd (5th if missed)	AJIO ads / TOT gap	worker cron
9:30 AM daily	Morning report (yesterday's sale vs last week, drop alerts)	GitHub schedule
10:00 AM daily	Myntra courier OTPs; Myntra daily reports; stock-out alert	worker cron / GitHub
10:15 AM Monday	Weekly review	GitHub schedule
11:00 AM daily	Myntra OTP retry	worker cron
11:00 AM on the 1st / 2nd	Dead stock (1st); monthly MIS and plan (2nd)	GitHub schedule
11:30 AM daily	Listing health	GitHub schedule
12:00 PM and 3:00 PM	SLA warning (orders due, not ready to ship)	GitHub schedule
2:00 PM on the 15th	Return analysis	worker cron
3:30 PM daily	Daily sale report	worker cron
5:30 PM daily	Myntra VMS; AJIO TAT	worker cron
6:00 PM daily	Label check (ERP labels vs OMSGuru)	GitHub schedule
6:30 PM daily	Myntra courier scorecard	GitHub schedule
8:00 PM daily	Evening summary	GitHub schedule
Every 3 hours (2, 5, 8, 11 AM / PM)	OMSGuru inventory push (watched only on failure)	GitHub schedule

Worker cron times live in worker/src/worker.js (SCHEDULE) and worker/wrangler.toml ([triggers], at most 5 on the free plan). GitHub schedule times live in each workflow's "schedule:".

5. Using Ultron: the owner's procedure

5.1 Asking

- Write in plain words. One question or request per message works best.
- Give exact numbers where they matter: order numbers, dates ("1 to 7 Oct"), account ("PPMP").
- Commands: `/start` shows what Ultron can do; `/dashboard` sends a sign-in link valid 10 minutes. The old menu commands (/sale, /vms ...) still work: they go to Ultron, which proposes the same report.

You write	Ultron does
how many orders yesterday?	reads OMSGuru, answers per channel with cancelled units
where is order FN2511505146?	finds it in OMSGuru: status, dates, SLA, courier, AWB, history
which orders are due today?	lists orders whose SLA is today or passed and are not ready to ship
why did the AJIO TAT run fail?	reads that run's log and explains
run the sale report for 1 to 7 Oct	sends Approve / Reject for sale with from / to
add FN123, FN456 to the TAT master	sends Approve / Reject for ajiotat with those orders

5.2 Approving

1. Read the summary and the exact run and inputs in the Approve message.
2. Tap Approve to start it, or Reject. One tap decides it; a second tap does nothing.
3. The report arrives like a scheduled one. Unanswered approvals expire after 24 hours.

*Be careful with **ajiospf** (raises claims and posts replies on AJIO) and **invpush**: approve a dry run first when unsure.*

5.3 Teaching and correcting

- Tell it lasting things: "remember: Snapdeal sales are not reported". It confirms with a note id.
- Ask "what do you remember about SPF?" to see notes; "forget a1b2c3" drops one.
- Tell the watcher what is harmless: "the 3:30 PM login drop in sale is OMSGuru maintenance, ignore it".
- Never send passwords or OTPs to the bot; the memory refuses them anyway.

5.4 Alerts

An alert names the bot, what happened, the cause, what was affected and the next step, with a link to the run. Do the suggested step or ask Ultron about it ("rerun it" gives an Approve).

6. Set-up from scratch

Use this when moving to a new PC, a new GitHub or Cloudflare account, or after a reset. Allow about an hour.

6.1 Prerequisites

- Windows, macOS or Linux PC with Python 3.12+, Git, Node.js 20+ and the GitHub CLI (gh).
- Accounts: GitHub (repo bindalbhikan/Ultron), Cloudflare (Workers + KV), Telegram bot token, Google service account with access to the Drive sheets, and the portal logins.

6.2 Get the code and local settings

```
git clone https://github.com/bindalbhikan/Ultron.git AGENT-ULTRON-BINDAL
cd AGENT-ULTRON-BINDAL
python -m venv .venv
.venv\Scripts\pip install -e ".[full,workings]"      (macOS / Linux: .venv/bin/pip)
copy .env.example .env      then fill in every value
put service_account.json (the Google key) next to agent.toml
ultron check
```

`ultron check` must show the company, repo bindalbhikan/Ultron, the 20 actions and the tools. .env and service_account.json are git-ignored: never commit them.

6.3 GitHub secrets and variables

In the Ultron repo, Settings > Secrets and variables > Actions, add every secret and variable listed in the Appendix. From the PC, each secret can be set without showing it on screen:

```
gh secret set NAME -R bindalbhikan/Ultron      (paste the value, Enter, then Ctrl+Z /
Ctrl+D)
gh secret set GOOGLE_SERVICE_ACCOUNT_JSON -R bindalbhikan/Ultron < service_account.json
gh variable set NAME -R bindalbhikan/Ultron --body "value"
```

6.4 Telegram worker

1. GitHub > Settings > Developer settings > Fine-grained tokens: a token for repository Ultron with **Actions: Read and write**.
2. In the worker folder run `powershell -ExecutionPolicy Bypass -File .\setup.ps1`: it deploys the worker, stores its secrets (TELEGRAM_BOT_TOKEN, GITHUB_TOKEN, WEBHOOK_SECRET, ALLOWED_CHAT_IDS, DASHBOARD_TOKEN, SESSION_KEY) and points the bot's webhook at it.
3. After any later code change: `npx wrangler deploy` in the worker folder.
4. The KV namespace "DASH" (id in wrangler.toml) holds dashboard numbers, Myntra logins, pending approvals and Ultron's memory.

6.5 Dashboard

In dashboard/: `npx wrangler deploy`. It reads the same KV namespace.

6.6 Verify

1. Send /start to the bot: the Ultron welcome appears and the old menu keyboard disappears.
2. Ask "how many orders came yesterday?": an answer arrives in 1 to 2 minutes.
3. Ask "run the SLA warning now", tap Approve: the SLA report arrives.
4. Teach one note and ask "what do you remember?": the note is listed (memory works).
5. In the repo's Actions tab, the next scheduled runs appear under bindalbhikan/Ultron.

7. Running Ultron on your own machine

Instead of GitHub Actions and the worker, the agent can run as one always-on process (`ultron serve`) on a PC, a server / VPS, a Mac or an Android phone. It polls Telegram itself, so nothing needs to be opened on the network, and keeps approvals itself. A Telegram bot is either polled or has a webhook: use a separate bot for this, or remove the worker's webhook first.

System	Set-up (in the folder with agent.toml and .env)	Runs as
Linux PC / VPS	sh deploy/linux/install.sh	systemd user service
macOS	sh deploy/macos/install.sh	launchd agent
Windows	powershell -ExecutionPolicy Bypass -File deploy\windows\install.ps1	scheduled task
Android (Termux)	sh deploy/termux/setup.sh (add --fallback for a stand-by phone)	termux-services + Termux:Boot
Docker	deploy/docker/Dockerfile	container

- Each one restarts Ultron when it stops and starts it with the machine.
- Fallback: run the main host normally and a phone with --fallback; the phone answers only while the main host is down (checks every 2 minutes).
- On a phone keep it charging and allow Termux to run in the background, or Android stops it.

Owner commands in this mode: /memory [words], /forget <id>, /new (fresh conversation).

8. Changing things

Change	Where	Then
A password or login	.env (local) and the GitHub secret of the same name	gh secret set NAME -R bindalbhikan/Ultron
A new Myntra login (session expired)	python scripts/myntra/myntra_session.py	saved to the worker; announced on Telegram
What Ultron may run	agent.toml [actions] AND worker/wrangler.toml [vars.AGENT_ACTIONS]: same workflow and inputs	commit, push, npx wrangler deploy (worker)
Ultron's company text, channels, tools, memory size	agent.toml	commit, push
A worker-cron time	worker/src/worker.js SCHEDULE (+ wrangler.toml [triggers] if a new hour)	npx wrangler deploy
A GitHub-schedule time	that workflow's schedule: cron (UTC = IST - 5:30)	commit, push
The model	repo variable AGENT_MODEL (an allowed Workers AI model)	next run uses it

8.1 Adding a new report

1. Code in src/ultron/workings/<name>/pipeline.py with main(argv) using run_bot; register it in run.py.
2. A workflow .github/workflows/<name>.yml that runs scripts/github/ci_run.sh <name> and uploads secure/ encrypted.
3. Add the workflow's name to agent-watch.yml so Ultron checks it; write docs/bots/<name>.md (the watcher reads it).
4. Optional: an action in agent.toml and wrangler.toml so the owner can ask for it; a schedule.

8.2 Releasing

Work in the local folder, commit, push to main. Workflows always run the latest main. Tag a release (git tag v0.x.y, git push origin v0.x.y) when a version is worth keeping.

9. Monitoring and troubleshooting

Where to look: the Telegram alert (it links the run); the repo's Actions tab; a run's full log with `python scripts/github/run_output.py <run id>` (needs ARTIFACT_KEY in .env); the worker's logs with `npx wrangler tail` in the worker folder.

Symptom	Likely cause	Fix
"The agent did not start" on Telegram	worker's GitHub token lacks Actions write on Ultron, or expired	new fine-grained token; wrangler secret put GITHUB_TOKEN
"the agent's model is not available"	Workers AI free daily allocation used up, or model down	wait for the next day; or set AGENT_MODEL to another allowed model
No reply at all	worker not deployed / webhook lost / chat not in ALLOWED_CHAT_IDS	npx wrangler deploy; run setup.ps1 or set_webhook.ps1; check the secret
"Memory not read" in a run log	the worker running is older than the memory route	npx wrangler deploy in the worker folder
Myntra report: session expired	the partner-portal login cookies ran out	python scripts/myntra/myntra_session.py
A report did not run at its time	worker cron (wrangler triggers) or GitHub schedule delay	ask Ultron to run it (Approve); check SCHEDULE / the workflow's cron
"... did not start" from an auto run	GitHub refused the dispatch (token, workflow name, inputs)	the message shows GitHub's reply; fix the token or the workflow
Telegram 409 with ultron serve	the bot has a webhook (a worker answers it)	use a separate bot for serve
git fetch: bad object refs/desktop.ini	Google Drive put desktop.ini into .git	delete .git/refs/**/desktop.ini
npm install leaves empty files	npm cannot extract packages on Google Drive	not needed: the worker imports ../agent.js from this repo

10. Security rules

- The repository is **public**: never commit .env, service_account.json, cookies, captures, Excel / CSV data or customer details (all are git-ignored). Business data stays on Drive.
- Secrets live only in GitHub secrets, the worker's secrets and the local .env. GitHub never shows a secret again: keep the local .env as the master copy.
- Run logs and outputs are uploaded only encrypted with ARTIFACT_KEY.
- Only chats in ALLOWED_CHAT_IDS / EXTRA_CHAT_IDS are answered; the webhook checks Telegram's secret header.
- Ultron's tools only read. Every change goes through Approve, limited to the listed workflows and inputs, once, within 24 hours.
- Text from portals, logs and files is treated as data, never as instructions to Ultron.
- The memory refuses passwords, OTPs, tokens, keys, phone numbers and emails.
- Before selling Ultron to other sellers: make the workings seller-neutral, move the repo to a company GitHub organisation, consider making it private, and get the name "Ultron" checked for trademark.

11. Retiring bindal-bots: checklist

#	Step	Status (25 Oct 2026)
1	All code, workflows, docs, worker and dashboard in bindalbhikan/Ultron	Done
2	GitHub secrets (33) and variables (9) in the Ultron repo	Done
3	Live check: Ultron answers from the Ultron repo (OMSGuru + model + Telegram)	Done
4	Worker's GitHub token: Actions Read and write on the Ultron repo	Owner to do
5	Deploy the worker from the Ultron folder (npx wrangler deploy)	Owner to do
6	/start on the bot; one question; one approved report; one scheduled run seen in Ultron's Actions	After 5
7	A day of scheduled reports arrives from the Ultron repo	After 5
8	Archive, then later delete, the bindal-bots repository and its local folder	Owner, after 7

Appendix A. Secrets, variables and folder layout

A.1 GitHub secrets (names only)

Group	Secrets
Telegram, worker, logs	TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID, DASHBOARD_URL, DASHBOARD_TOKEN, ARTIFACT_KEY
Google Drive	GOOGLE_SERVICE_ACCOUNT_JSON, DRIVE_RELAY_URL, DRIVE_RELAY_TOKEN, SALE_DRIVE_FILE_ID, VMS_DRIVE_FILE_ID, TAT_DRIVE_FILE_ID, TAT_DRIVE_FOLDER_ID, SJIT_DRIVE_FOLDER_ID, RETURN_DRIVE_FOLDER_ID, SPF_MASTER_DRIVE_FILE_ID, SPF_OMS_DRIVE_FILE_ID, SPF_MANUAL_DRIVE_FILE_ID, MYNTRA_SPF_FOLDER_ID, AJIO_ADS_FOLDER_ID
Portals	OMSGURU_EMAIL, OMSGURU_PASSWORD, AJIO_USERNAME, AJIO_PASSWORD, FLIPKART_EMAIL, FLIPKART_PASSWORD, ERP_USERNAME, ERP_PASSWORD, ERP_USER_TOKEN, SRM_USERNAME, SRM_PASSWORD, VMS_USERNAME, VMS_PASSWORD
Other	GEMINI_API_KEY; MYNTRA_SESSION_JSON (optional fallback: logins are kept on the worker)

Worker secrets (wrangler secret put): TELEGRAM_BOT_TOKEN, GITHUB_TOKEN, WEBHOOK_SECRET, ALLOWED_CHAT_IDS, EXTRA_CHAT_IDS, DASHBOARD_TOKEN, SESSION_KEY.

A.2 Repository variables

AJIO_CHANNEL_IDS, FBF_LOCATIONS, MYNTRA_CHANNELS, MYNTRA_SPF_TICKETS, SALE_REPORT_DAYS, SLA_LOOKBACK_DAYS, SPF_DRY_RUN, TAT_LOOKBACK_DAYS, VMS_REPORT_DAYS; optional AGENT_MODEL.

A.3 Folder layout

Path	What it is
agent.toml	Bindal's Ultron set-up: company text, channel names, actions, tools, memory
src/ultron/core, agents, tools, portals, integrations, host	the agent: config, model loop, memory, approvals, chat, watch, tools
src/ultron/workings/	the report workings and their portal clients
run.py	python run.py <report>: what each report workflow runs
.github/workflows/	one workflow per report, plus Agent watch and Agent chat
worker/	the Telegram worker (src/worker.js, wrangler.toml) and Ultron's routes (agent.js)
dashboard/	the dashboard worker
scripts/	Myntra login, run log reader, CI runner, Drive relay, local run
docs/	this SOP, how each report works (docs/bots), architecture, portals
deploy/, examples/, main.py	running ultron serve as a service; every agent.toml key; run without installing